5 Array Handling

Arrays First

Write a program called **ArraysFirst** that initialises an array of seven integers. The array should contain the following values in the given order: 6, -4, 5, 3, -1, 0, and 8. First, the program should iterate the array and print all positive values on a single line. Next, the program should iterate the array again and print all negative values on the next line of the output.

Example output:

```
6 5 3 8
-4 -1
```

Arrays Iteration

Write a program called **ArraysIteration** that first inputs five integers from the user and *saves them in an array*. Then, the program should print the integers *in descending order* on a single line.

Example output:

```
Enter an integer: 3
Enter an integer: 9
Enter an integer: 1
Enter an integer: 4
Enter an integer: 7

9 7 4 3 1
```

Arrays Letters

Write a program called **ArraysLetters** that first inputs a grade letter from user. Then, the program should compute and show the percentage of the grade letter found in an array that contains the following values: "A", "A", "B", "A", "C", "B", "C", "F", "B", "B", "A", "C", "C", "C"

Example output:

```
Enter letter: A
28,6 %
```

Arrays Translation

Write a program called **ArraysTranslation** that inputs an English word from the user and translates it to Finnish. This time, the program can translate only the following words:

"bus", "car", "cab", "house", "moon", "sun", "thanks", "train", "teacher"

If the English word is unknown, the program should display "**Unknown word**". Use two arrays in the program.

Example output:

```
Enter an English word: teacher
opettaja
```

Arrays Candidates

Write a program called **ArraysCandidates**. First, the program inputs the number of candidates. You can suppose that there is always at least one candidate. Then, the program should input name and the number of votes for each candidate. Finally, the program should determine the winner and display the winner's name and votes.

Example output:

```
Enter the number of candidates: 2
Enter name: Ed
Enter Ed's votes: 45
Enter name: Ann
Enter Ann's votes: 46

Ann is the winner with 46 votes!
```

If the election is a tie, then the program should print the names and votes of the candidates who share the highest votes.

Example output:

```
Enter the number of candidates: 4
Enter name: Cathy
Enter Cathy's votes: 100
Enter name: Ann
Enter Ann's votes: 100
Enter name: Ben
Enter Ben's votes: 99
Enter name: Frank
Enter Frank's votes: 100

It is a tie!
Cathy (100 votes)
Ann (100 votes)
Frank (100 votes)
```

Arrays Matrix (Handling two-dimensional arrays)

Write a program called **ArraysMatrix**. First, copy/paste the main method below into your program class. Then, write the ***printMatrixSum*** method. The *printMatrixSum* method should print the sum of two matrices.

```
public static void main(String[] args) {
    int[][] m1 = { {1, 2, 0}, {2, 3, 4} };
    int[][] m2 = { {3, 2, 5}, {6, 4, 3} };
    int[][] m3 = { {1, 1, 1, 1}, {3, 2, 3, 1}, {2, 2, 2, 2} };
    int[][] m4 = { {2, 2, 3, 3}, {2, 3, 1, 0}, {1, 2, 3, 4} };

    printMatrixSum(m1, m2);
    System.out.println();

    printMatrixSum(m3, m4);
}
```

The example output of the program is as below.

Example output:

```
4 4 5
8 7 7

3 3 4 4
5 5 4 1
3 4 5 6
```

INSTRUCTION

In Java, we can Write *multi-dimensional arrays*. The code in the *main* method above creates four ***two-dimensional arrays***. A two-dimensional array (***matrix***) resembles a table with rows and columns. We can access the values in a two-dimensional array with a 'double-index'.

first

	0	*1*	*2*
0	**1**	**2**	**0**
1	**2**	**3**	**4**

```
System.out.println(first[0][0]);    // Prints 1  (row 0, column 0)
System.out.println(first[1][2]);    // Prints 4  (row 1, column 2)
```

second

	0	*1*	*2*
0	**3**	**2**	**5**
1	**6**	**4**	**3**

```
System.out.println(second[0][1]);    // Prints 2  (row 0, column 1)
System.out.println(second[1][0]);    // Prints 6  (row 1, column 0)
```

We can get the *number of **rows*** in a two-dimensional array as follows:

```
System.out.println(first.length);          // Prints 2
```

We can get the *number of **columns*** in a two-dimensional array as follows:

```
System.out.println(first[0].length);       // Prints 3
```

NOTE! Two matrices must have an equal number of rows and columns to be added. The example below shows an ***entry-wise addition of two matrices***.

$$\begin{bmatrix} 1 & 2 & 0 \\ 2 & 3 & 4 \end{bmatrix} + \begin{bmatrix} 3 & 2 & 5 \\ 6 & 4 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 4 & 5 \\ 8 & 7 & 7 \end{bmatrix}$$

Arrays Deviation (*grade level 4 – 5 exercise*)

First, write a class called **ArraysLibrary** for storing *generic reusable methods*. This class is a *method library class* and it does not have the main method. Instead, the class should have the following methods: ***max***, ***min***, ***median***, ***mean***, and ***standardDeviation***. Each method should take an *array of integers* as parameter and return the appropriate result.

Then, write a program called **ArraysDeviation** that uses methods of the **ArraysLibrary** class. First, the main method should ask the user how many values there are in the sample. Then, the main method should input the values from the user and save them to an array. Finally, the program should compute the required descriptive statistics from the sample data and display them. See the example output for more details.

NOTE! The main method should call the methods ***max***, ***min***, ***median***, ***mean***, and ***standardDeviation*** of the ***ArraysLibrary*** class to get the values to be printed.

NOTE! The *minimum allowed sample size is 4*. If the user enters less than 4 as the number of values, then the program should just print the text "Sorry, at least 4 values required".

Example output:

```
Enter the number of values: 5
Enter an integer: 6
Enter an integer: 54
Enter an integer: 1
Enter an integer: 43
Enter an integer: 3

n = 5
Min: 1
Max: 54
Mean: 21,4
Median: 6,0
Sample standard deviation: 25,1
Sample data: 6 54 1 43 3
```

The ***sample standard deviation formula*** is the following:

$$s = \sqrt{\frac{\sum_i (x_i - \bar{x})^2}{n-1}}$$

s	= sample standard deviation	$\sum$	= sum of ...
x_i	= individual value in the sample	$\bar{x}$	= sample mean
n	= number of values in the sample.		

1. First, compute the sample mean. NOTE! For this purpose, call the ***mean*** method of your **ArraysLibrary** class and assign the sample mean to a variable.
2. Then, for each value: subtract the sample mean and square the result. Compute the sum of those squared differences.

3. Next, divide the sum of the squared differences by (n - 1)
4. Finally, use the **Math.sqrt** method to compute the square root of the quotient of the division.

The ***median*** is the middle value in a sorted list of numbers. Examples:

4, 7, 8 *median* = 7 1, 3, 4, 6 *median* = (3 + 4) / 2 = 3.5

Arrays Cities (*grade level 4 – 5 exercise*)

Write a program called **ArraysCities**. The program should first input the number of cities to be saved to an array. Then, the program should create the array, input cities, and save them to the array. Finally, the program should find all ***distinct city names*** in the array and print them in alphabetical order. See the example output for more details.

NOTE! This is an exercise on ***generic*** *algorithm design*. Therefore, you should not create another array or collection in your code. This time, you should tackle the problem with ***one array***.

Example output:

```
Enter the number of city names: 7
Enter city name: Helsinki
Enter city name: Helsinki
Enter city name: Espoo
Enter city name: Espoo
Enter city name: Vantaa
Enter city name: Espoo
Enter city name: Vantaa

Espoo Helsinki Vantaa
```

Arrays Lotto (*grade level 4 – 5 exercise*)

NOTE! This exercise cannot be tested in Viope. Therefore, it available as an "open" exercise in Viope. Please write and test this program in Eclipse. Then, copy/paste your code to Viope and submit it.

Write a program called **ArraysLotto** that generates 7 unique lotto numbers (no duplicate numbers allowed) and shows them in ascending order. Each lotto number is a random integer between 1 and 39.

```
Console   Problems
<terminated> ArraysLottoNumbers [Java Application]
4  7  12  15  22  28  30
```

You can generate pseudo-random integers as below. NOTE! You should create *exactly one* random generator in the ArraysLotto program.

```
Random randomGenerator = new Random();
int x = randomGenerator.nextInt(5);      // Generates an integer between 0 and 4
```